# xlog: A GPU-Native Logic Programming Language for Unified Symbolic Reasoning

The xlog Project

v0.6.1 Technical Whitepaper

April 2026

**Abstract**

We present xlog, a GPU-native logic programming language designed for unified symbolic reasoning. The language provides typed predicates, user-defined functions, modules, arithmetic, stratified aggregations, and integrity constraints, plus probabilistic facts and neural predicate declarations as first-class syntactic constructs. Programs compile to four reasoning backends on a single CUDA runtime: deterministic Datalog evaluation via semi-naive fixpoint, probabilistic inference via knowledge compilation (PIR $\rightarrow$ CNF $\rightarrow$ D4 $\rightarrow$ XGCF), SAT/MaxSAT verification, and differentiable neural-symbolic training—with zero host–device transfers in production paths. xlog is implemented in Rust with 22 custom CUDA kernel files (14.4K lines of device code) and provides zero-copy interoperability with PyTorch, JAX, and columnar analytics frameworks via DLPack and Arrow interfaces. Circuit caching across training iterations yields a measured 2.74× end-to-end speedup (95% CI: $[2.29, 3.18]$) on the MNIST addition benchmark.

## Contents

# 1 Introduction

Symbolic AI and neural AI have followed divergent engineering paths. Datalog engines, probabilistic logic systems such as ProbLog, and inductive logic programming (ILP) frameworks are implemented as CPU-bound interpreters or compilers, processing relations and proofs in main memory. Deep learning frameworks—PyTorch, JAX—execute dense tensor computations on GPUs via highly optimized CUDA kernels. When researchers combine the two paradigms, as in DeepProbLog [1] or NeurASP [2], the symbolic component remains on the CPU while the neural component runs on the GPU. Every training iteration transfers data across the PCIe bus: the CPU-side logic engine materializes query results, ships them to the GPU for gradient computation, then pulls gradients back to update symbolic parameters. At scale—millions of ground atoms, thousands of training steps—these host–device transfers dominate wall-clock time and memory bandwidth, becoming the primary bottleneck rather than the inference or learning computation itself.

The gap is architectural. Existing systems address individual reasoning tasks on the GPU in isolation: GPU-accelerated Datalog evaluation [3, 4], GPU SAT solvers [5], or differentiable logic on CPU with GPU-side neural networks [6]. No single system performs deterministic Datalog evaluation, probabilistic inference via knowledge compilation, SAT/MaxSAT verification, and differentiable neural-symbolic training entirely on the GPU with zero host–device data transfers in production paths. The absence of such a platform forces practitioners into multi-system pipelines—a Datalog engine for rule evaluation, a separate probabilistic reasoner, a Python training loop bridging CPU logic to GPU tensors—each with its own data format, memory model, and failure modes.

xlog addresses this gap as a GPU-native logic programming language whose compiler and runtime span four reasoning paradigms: deterministic logic (semi-naive evaluation with stratified negation), probabilistic inference (exact knowledge compilation and Monte Carlo sampling), SAT/MaxSAT solving (GPU CDCL with proof certificates), and neural-symbolic learning (differentiable training with PyTorch interoperability). The system is implemented in Rust with 22 custom CUDA kernel files (14.4K lines of device code) organized into a layered crate architecture. The compilation pipeline transforms Datalog source into a relational intermediate representation (RIR), lowers probabilistic programs through a propositional intermediate representation (PIR) into CNF, compiles decision-DNNF circuits via D4 [7], and encodes the result in a GPU-resident circuit format (XGCF) for forward and backward evaluation. All semantic data structures—fact stores, circuit nodes, solver state, gradient buffers—remain GPU-resident during execution. Host involvement is limited to orchestration, I/O, and compilation.

The principal contributions of this paper are:

- **A typed logic programming language for GPU execution.** The `xlog-logic` crate implements a statically-typed logic language with user-defined functions, modules and imports, stratified aggregation, integrity constraints, probabilistic facts, and neural predicate declarations—all compiling through a single PEG parser, type inferencer, and SCC-based stratifier into the relational IR that feeds every downstream backend (Section 3).
- **GPU-resident semi-naive Datalog evaluation.** The `xlog-runtime` and `xlog-cuda` crates implement relational algebra operators (hash join, radix sort, filter, deduplication, set difference, grouped aggregation) as CUDA kernels, executing fixed-point iteration entirely on the GPU with columnar storage and HISA indexing [3].

- **GPU-native knowledge compilation pipeline.** The `xlog-prob` crate compiles probabilistic Datalog programs through a PIR-to-CNF-to-D4-to-XGCF pipeline, producing GPU-resident arithmetic circuits with compile-once/evaluate-many semantics. Forward and backward passes over the circuit run as level-parallel CUDA kernels, enabling exact weighted model counting and gradient computation without host transfers.
- **End-to-end differentiable neural-symbolic training.** The `xlog-neural` crate and `pyxlog` Python package connect compiled circuits to PyTorch's autograd graph. Circuit structure depends on the logic program, not on neural network weights, so compiled XGCF templates are cached across training iterations. This circuit caching yields a measured 2.74× end-to-end training speedup (95% CI: $[2.29, 3.18]$) by eliminating redundant D4 recompilation.
- **Zero-copy interoperability with ML frameworks.** The `xlog-cuda` crate exposes GPU-resident query results and gradient tensors via DLPack [8] capsules and Arrow [9] IPC/C Data interfaces, enabling direct consumption by PyTorch, cuDF, and other frameworks without data copies or device synchronization.
- **Differentiable ILP with GPU-resident credit assignment.** The `pyxlog` dILP trainer implements sparse GPU mask computation, a fully device-resident credit/loss path with zero host transfers, and a six-gate promotion pipeline (convergence, novel-rate audit, regression check, holdout F1 threshold, ambiguity scan, typed schema validation) for transactional rule induction. This subsystem is currently in beta.

The remainder of this paper is organized as follows. Section 2 presents the system architecture and crate decomposition. Section 3 describes the xlog language surface. Section 4 details GPU-native deterministic Datalog evaluation. Section 5 covers the probabilistic inference pipeline. Section 6 details the neural-symbolic bridge and differentiable training. Section 7 discusses interoperability with external frameworks. Section 8 presents evaluation results. Section 9 surveys related work. Section 10 discusses limitations and future directions.

# 2 Architecture

## 2.1 System Overview

xlog's current workspace contains 15 Rust crates organized around a five-tier product/runtime hierarchy. Figure 1 summarizes the primary dependency spine and certification surface; two support crates discussed below (`xlog-induce` and `xlog-integration`) are omitted from the diagram for readability. The architecture still enforces strict layering: no crate depends on a crate in its own tier or a higher tier.

**Tier 0** contains `xlog-core`, the leaf crate that defines shared scalar types (`ScalarType`, `Schema`, `AggOp`), the `KernelProvider` trait, error types, memory budgets, runtime configuration, and a bidirectional `SymbolTable` for dictionary-encoded strings. Every other crate in the workspace depends on `xlog-core`.

**Tier 1** provides domain-specific intermediate representations and service providers that depend only on `xlog-core`. `xlog-ir` defines the relational IR node tree (`RirNode`), expression algebra, and `ExecutionPlan` structures. `xlog-cuda` wraps CUDA device management via cudarc [10], stages

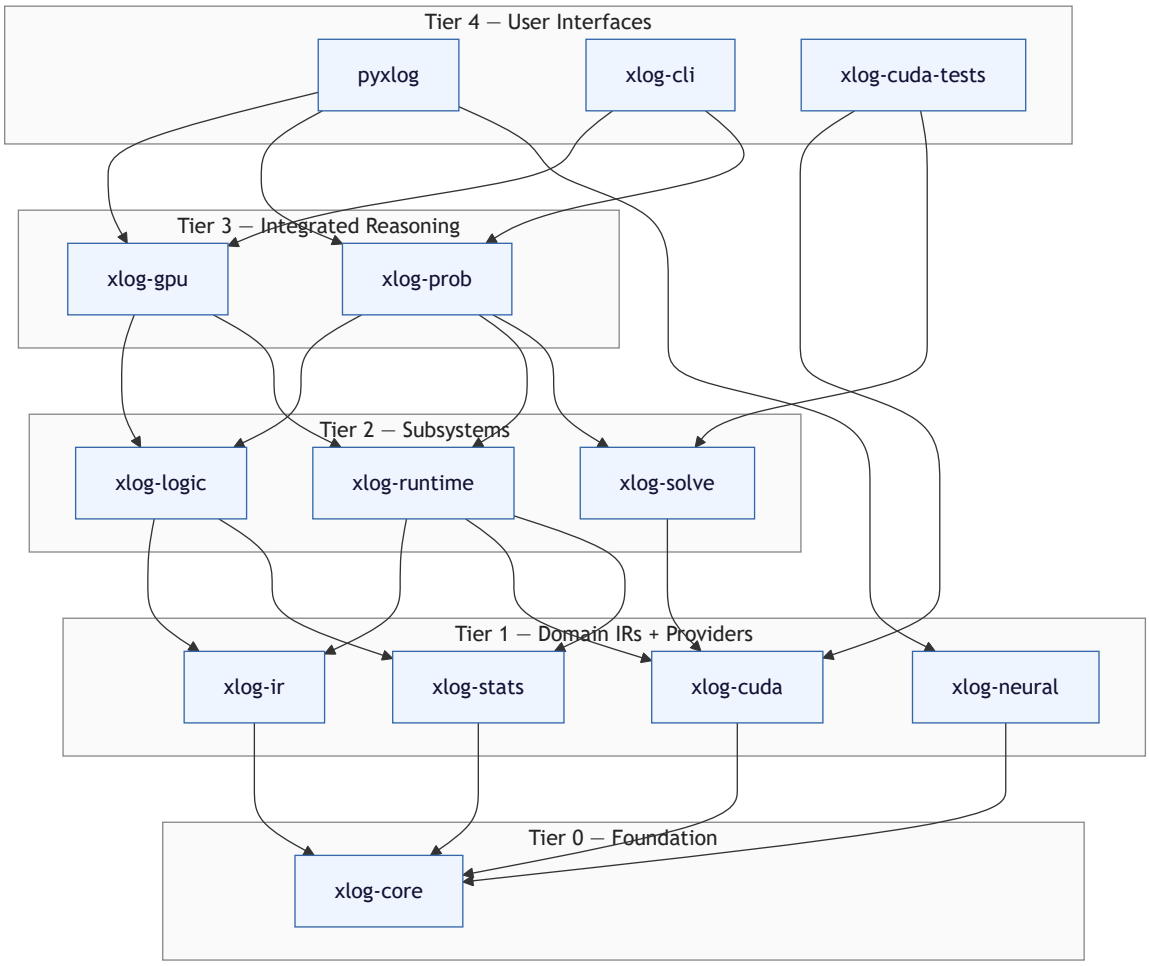


Figure 1: xlog crate hierarchy. Tier 0 is the leaf foundation; each higher tier depends only on strictly lower tiers.

CUDA kernel modules (cubin-first with portable PTX fallback), implements the `CudaKernelProvider` with modular submodules (relational, filter, group-by, arithmetic, I/O, ILP, probabilistic, transfer, kernel loading), and exposes Arrow IPC/C Data and DLPack interoperability. `xlog-stats` provides `StatsManager` for compiler feedback and runtime tracking. `xlog-neural` defines the `NetworkRegistry`, `NetworkHandle`, `TensorSourceRegistry`, `NeuralBridge`, and `BatchCollector` types for neural-symbolic integration, with an optional PyO3 [11] feature gate for Python interop.

**Tier 2** implements the three core subsystems. `xlog-logic` implements the xlog language frontend (see Section 3): PEG parser (pest), SCC-based stratifier, AST-to-RIR lowerer, and cost-aware optimizer. An internal rule-rewriting pass in `expand.rs` (993 lines) normalizes repeated patterns before stratification; this pass is invisible to user programs. xlog programs may also carry `#pragma` directives (e.g., `prob_engine`, `prob_cache`, `max_recursion_depth`) that influence compiler behavior; pragmas parse into dedicated AST nodes consumed by the compiler before lowering. Its production dependencies are `xlog-core`, `xlog-ir`, and `xlog-stats`. `xlog-runtime` provides the `Executor` (modular across delta computation, expression evaluation, join caching, node dispatch, recursive evaluation, and rewriting), versioned `RelationStore`, profiling, and query statistics. It depends on `xlog-core`, `xlog-ir`, `xlog-cuda`, and `xlog-stats`. `xlog-solve` implements GPU CDCL verification (complete SAT/UNSAT with on-GPU model and proof validation) and continuous local search for SAT/MaxSAT. It depends on `xlog-core` and `xlog-cuda`.

**Tier 3** composes Tier 2 subsystems into integrated reasoning pipelines. `xlog-gpu` provides a high-level deterministic execution API with input/output buffer management; it depends on `xlog-core`, `xlog-cuda`, `xlog-ir`, `xlog-logic`, and `xlog-runtime`. `xlog-prob` implements the full probabilistic tier—provenance tracking, PIR-to-CNF encoding, GPU-native D4 compilation, XGCF circuit construction, exact inference, Monte Carlo sampling, and circuit caching—depending on `xlog-core`, `xlog-cuda`, `xlog-ir`, `xlog-logic`, `xlog-runtime`, and `xlog-solve`. `xlog-induce` is a support subsystem for bounded exact induction on top of the runtime/CUDA stack; it depends on `xlog-core`, `xlog-cuda`, and `xlog-runtime`, and is consumed by the Python ILP surface.

**Tier 4** presents user-facing interfaces. `pyxlog` is a PyO3 extension module that bridges nine internal crates (`xlog-core`, `xlog-cuda`, `xlog-gpu`, `xlog-prob`, `xlog-logic`, `xlog-neural`, `xlog-runtime`, `xlog-ir`, `xlog-induce`) to Python, exposing DLPack-first evaluation, training APIs, and ILP/dILP trainers. `xlog-cli` is the published Cargo package that provides the `xlog` command-line binary for deterministic and probabilistic execution with Arrow IPC I/O.

Outside the product/runtime stack, `xlog-cuda-tests` is an unpublished certification suite for GPU release gating, and `xlog-integration` hosts workspace-only cross-crate executor tests and integration binaries.

### 2.2 Compilation Pipeline

The compilation pipeline transforms Datalog source text into a GPU-executable plan through five stages: parsing, stratification, lowering, optimization, and plan assembly.

**Parsing.** The PEG parser in `crates/xlog-logic/src/parser.rs` uses the pest parser generator with a grammar defined in `grammar.pest`. It produces an abstract syntax tree (AST) containing `Program`, `Rule`, `Atom`, `Term`, `BodyLiteral`, and related nodes defined in `crates/xlog-logic/src/ast.rs`. The parser handles the full xlog language surface (Section 3), including probabilistic facts, annotated disjunctions, neural predicate declarations, user-defined functions, and module imports.

**Stratification.** The stratifier in `crates/xlog-logic/src/stratify.rs` builds a predicate dependency graph with three edge types—positive, negative, and aggregate—then computes strongly connected components (SCCs) using Tarjan's algorithm. If any SCC contains a negative or aggregate edge, the program is rejected with a diagnostic error: stratified negation and stratified aggregation require that no recursion passes through negation or aggregation. The output is an ordered sequence of strata, each containing one or more SCCs that can be evaluated before any stratum that depends on them.

**Lowering.** The `Lowerer` in `crates/xlog-logic/src/lower.rs` translates each AST rule into a `RirNode` tree defined in `crates/xlog-ir/src/rir.rs`. Body literals become `Scan` nodes; variable bindings across atoms produce `Join` nodes with key columns derived from shared variable positions; negated literals become `Anti` joins; filters and arithmetic constraints become `Filter` nodes with `Expr` predicates (supporting comparison, Boolean connectives, arithmetic, built-in functions, and conditional expressions). The lowerer also emits `Fixpoint` nodes that wrap recursive SCC bodies for semi-naive iteration, and `GroupBy` nodes for stratified aggregation.

**Optimization.** The optimizer in `crates/xlog-logic/src/optimizer.rs` applies cost-aware transformations using runtime statistics from `xlog-stats::StatsManager`. It performs predicate pushdown—relocating `Filter` nodes below `Project` and `Join` nodes to reduce intermediate result sizes—and cost-based join ordering via dynamic programming. For queries involving more relations than a configurable threshold (`dp_threshold`, default 10), the optimizer falls back to a greedy heuristic to avoid exponential planning time. Cost estimates account for expected row counts, GPU memory consumption, and data transfer volume.

**Plan assembly.** The final output is an `ExecutionPlan` (defined in `crates/xlog-ir/src/plan.rs`) containing SCCs in dependency order, strata for negation ordering, and compiled rule trees grouped by SCC. The `xlog-runtime` executor interprets this plan by dispatching each `RirNode` to the appropriate CUDA kernel via the `CudaKernelProvider`.

### 2.3 GPU Residency Model

xlog enforces a hard guarantee: all runtime semantic state resides in GPU device memory, or the system returns a deterministic error. There is no silent out-of-core spilling, and there is no implicit CPU fallback. If a workload exceeds the GPU memory budget, xlog raises a `RESOURCE_EXHAUSTED` error with diagnostic information rather than degrading to host memory transparently.

The residency contract extends to production query paths, where the system targets zero device-to-host (D2H) transfers. The `CudaKernelProvider` in `crates/xlog-cuda/src/provider/mod.rs` implements byte-level transfer accounting through a `HostTransferStats` structure that tracks `dtoh_bytes` and `htod_bytes` via atomic counters. Callers invoke `reset_host_transfer_stats()` before a performance-critical section and `host_transfer_stats()` afterward to obtain a snapshot. A separate `d2h_transfer_count` atomic counter increments once per `download_column_*` call, enabling assertions—particularly in the ILP trainer—that no column downloads occurred during a device-resident computation. The `reset_d2h_transfer_count()` method zeroes this counter for bracketed verification.

This design matters for performance. PCIe 4.0 $\times$16 delivers roughly 32 GB/s peak bidirectional bandwidth, while GPU HBM provides 1–3 TB/s depending on the device. A single unnecessary D2H round-trip for a moderate relation (e.g., 10M tuples at 16 bytes each, roughly 160 MB) costs approximately 5 ms of bus transfer latency—enough to dominate a semi-naive iteration step that

completes in microseconds on-device. By keeping fact stores, delta relations, circuit node values, solver state, and gradient buffers exclusively on the GPU, xlog eliminates this class of bottleneck. Host involvement is restricted to orchestration (launching kernels, managing plan execution order), I/O (reading source files, writing final results), and compilation (parsing, stratification, lowering, and optimization all run on the CPU before execution begins).

As of v0.5.5, this accounting is enforced rather than advisory: a strict D2H guard rejects unexpected data-plane transfers in deterministic Datalog evaluation, with a single explicit carve-out for binary-join output counts—treated as control-plane metadata reads (PRs #49, #52). v0.6.0 layers a stream-safe device-runtime stack (`AsyncCudaResource` → `LoggingResource` → `GlobalDeviceBudget` → `XlogDeviceRuntime`) and a `LaunchRecorder` discipline (preflight + commit, access-aware cross-stream dependency tracking) underneath the kernel provider; operator surfaces expose recorded-launch siblings under env-gated dispatch (umbrella `XLOG_USE_RECORDED_OPS=1`). Architecture details live in `docs/architecture/device-runtime.md` and `docs/architecture/recorded-launch-migration.md`.

The `MemoryBudget` configuration in `xlog-core` already carries an `allow_ooc` flag, but current `main` treats it as reserved configuration rather than an implemented spill path. In today's runtime there is no silent out-of-core execution: workloads that exceed the GPU memory budget fail deterministically with `RESOURCE_EXHAUSTED`.

### 2.4 IR Stack

xlog uses a layered intermediate representation stack designed for extensibility across reasoning paradigms. Each IR level serves a distinct role in the compilation and evaluation pipeline.

**AST (Abstract Syntax Tree).** The parser produces a concrete syntax tree that preserves source-level structure: rules, atoms, terms, probabilistic annotations, neural predicate declarations, and directives. The AST lives in `crates/xlog-logic/src/ast.rs` and carries no execution semantics.

**RIR (Relational IR).** The lowerer transforms the AST into an algebraic tree of `RirNode` variants defined in `crates/xlog-ir/src/rir.rs`. The key variants are: `Scan` (base relation access), `Filter` (predicate evaluation), `Project` (column selection and computation via `ProjectExpr`), `Join` (inner, left-outer, semi, and anti joins with explicit key columns), `GroupBy` (stratified aggregation), `Union`, `Distinct`, `Diff` (set difference for semi-naive delta computation), and `Fixpoint` (recursive SCC wrapper). RIR nodes carry metadata including estimated cardinality ranges, memory peak estimates, skew hints for join partitioning, and incremental update semantics (delta vs. full materialization).

**PIR (Provenance IR).** For probabilistic programs, `xlog-prob` constructs a `PirGraph` (defined in `crates/xlog-prob/src/pir.rs`) whose `PirNode` variants—`Lit`, `NegLit`, `And`, `Or`, `Decision`—represent the weighted Boolean formula derived from provenance tracking over the ground program. PIR captures the probabilistic structure needed for knowledge compilation without encoding execution order.

**XGCF (xlog GPU Circuit Format).** The final compiled form is a levelized DAG stored in `crates/xlog-prob/src/xgcf.rs` as an `Xgcf` structure. Each node has a type (`Const0`, `Const1`, `Lit`, `And`, `Or`, `Decision`), child indices, and variable/literal identifiers. Level offsets enable parallel evaluation: all nodes at the same topological level execute in a single kernel launch. Forward passes compute log-space values; backward passes propagate adjoints for gradient computation. The

`GpuXgcf` structure in `crates/xlog-prob/src/gpu.rs` holds the device-resident buffers. Multiple circuits can be batched via `circuit_offsets` for fused evaluation across neural batch dimensions.

**Extensibility.** The IR stack was designed to accommodate additional representations as the system grows. The architecture documentation describes two planned IRs: **EIR** (Epistemic IR) for world-view reasoning with modal operators, split plans, and guess spaces; and **SIR** (Solver IR) for Boolean satisfiability encoding with CNF clauses, cardinality constraints, weight vectors, and proof policies. Neither EIR nor SIR is implemented in v0.6.1. The layered design ensures that adding a new IR requires implementing a lowering pass from RIR (or from another IR at the same level) and a backend that targets either XGCF or a new GPU-resident evaluation format, without modifying existing IR definitions or evaluation paths.

# 3 The xlog Language

This section describes the xlog language surface: types, expressions, user-defined functions, modules, and stratified aggregations with constraints. Probabilistic facts (`p::f.`) and neural predicate declarations (`nn/k`) are language-level constructs described in Section 5 and Section 6 respectively.

## 3.1 Design Philosophy

xlog is a typed logic programming language designed for GPU execution. Three principles shape its surface:

**Typed predicates.** Every predicate has a declared signature over a closed set of scalar types (`u32`, `u64`, `i32`, `i64`, `f32`, `f64`, `bool`, `symbol`). On current `main`, predicate schemas and lowering-time validation catch argument mismatches before any GPU kernel is generated; richer standalone type inference exists in reserved frontend APIs but is not yet the universal production entry point.

**Stratified semantics as contract.** Negation, aggregation, and recursion interact through a strongly-connected-component analysis on the predicate dependency graph. Programs whose SCCs cross negation or aggregation edges are rejected at compile time. Every accepted program has a well-defined fixpoint and a deterministic stratum order, which the runtime relies on for GPU scheduling.

**One language for four paradigms.** Probabilistic facts (`p::f.`), annotated disjunctions, neural predicate declarations (`nn/k`), and SAT constraints share the same syntactic core with deterministic Datalog. Parsing, AST construction, dependency analysis, and most lowering logic are shared across the workspace, while backend-specific expansion and compilation passes refine the resulting plans for deterministic, probabilistic, SAT, and neural-symbolic execution. The distinction between reasoning paradigms lives primarily in the backend, not in the surface syntax.

A minimal xlog program exhibits the core surface:

```
// The minimal xlog program: typed predicates, facts, a recursive rule, a query.
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
```

```
reach(X, Z) :-reach(X, Y), edge(Y, Z).

?-reach(1, N).
```

Listing 1: Minimal xlog program: typed predicates, facts, a recursive rule, a query.

Typed predicate declarations, ground facts, recursive rules, and a query appear in eight lines. The remainder of this section expands each component and adds the extensions that move xlog beyond classical Datalog.

### 3.2 Type System

Every predicate is declared with a typed signature via `pred`. Eight scalar types are supported: `u32`, `u64`, `i32`, `i64` for fixed-width integers; `f32` and `f64` for floats with IEEE 754 semantics; `bool`; and `symbol` for dictionary-encoded atoms. String literals in source (`"alice"`) are terms that map to `symbol` through the `SymbolTable` in `xlog-core`, giving the runtime dense integer identifiers while preserving source-level readability. Users can also introduce named type aliases through `domain` declarations.

Current `main` enforces type consistency through explicit predicate schemas plus frontend/lowering checks. Variable occurrences across body atoms must agree with declared predicate signatures; literal constants are validated against their destination column types; arithmetic expressions preserve scalar types in the emitted `Expr` tree. The dedicated pass in `crates/xlog-logic/src/typeinfer.rs` covers richer inference cases, especially around user-defined functions, but remains a reserved API rather than the sole wired frontend path.

A well-typed program:

```
pred node(u32, symbol).
pred connected(u32, u32).
pred bridge(u32, u32).

node(1, "alice"). node(2, "bob").
connected(1, 2).

bridge(A, B) :-connected(A, B), node(A, _), node(B, _).
```

Listing 2: A well-typed xlog program: typed bridge predicate.

Changing `bridge`'s declaration to `pred bridge(symbol, u32).` would cause inference to report that the head argument `A` is constrained at `symbol` (head) and `u32` (through `connected`), rejecting the program at compile time.

### 3.3 Expressions and Arithmetic

Rule bodies may contain arithmetic expressions, built-in function calls, and comparison filters. Arithmetic bindings use the `is` operator following Prolog convention: `D is X + Y` computes `X + Y` and binds the result to `D`. The right-hand side supports the standard arithmetic operators (`+`, `-`, `*`, `/`, `%`), the built-ins `abs`, `min`, `max`, `pow`, and `cast`, and calls to user-defined functions (Section 3.4).

Comparison literals (`<`, `<=`, `=`, `==`, `!=`, `>=`, `>`) compile to `Filter` RIR nodes and push down below joins where profitable.

Float arithmetic follows IEEE 754 with explicit NaN and $\pm\infty$ handling: `NaN != NaN`, and ordering predicates place `NaN` outside any other value rather than raising errors. This matters for aggregations, where a single unguarded NaN would contaminate downstream GPU computations.

A pairwise-proximity query illustrates expression use:

```
pred point(u32, f64, f64).
pred close(u32, u32).

point(1, 0.0, 0.0). point(2, 0.5, 0.5). point(3, 5.0, 5.0).

close(A, B) :-
  point(A, Xa, Ya),
  point(B, Xb, Yb),
  A < B,
  D is (Xa - Xb) * (Xa - Xb) + (Ya - Yb) * (Ya - Yb),
  D < 1.0.
```

Listing 3: Arithmetic in rule bodies: pairwise proximity via squared Euclidean distance.

The rule compiles to a self-join of `point` with an intermediate `Project` that materializes `D`, followed by a `Filter` on `D < 1.0`. The optimizer relocates the cheap comparison `A < B` ahead of the join when profitable, reducing intermediate cardinality.

### 3.4 User-Defined Functions

xlog includes a user-defined function (UDF) surface for pure scalar computations inside rule bodies and expressions. A UDF is introduced with the `func` keyword; parameters carry optional type annotations, and the return type is declared after a `->`. Arithmetic-expression bodies (optionally with conditional `if/then/else`) are the path integrated into the current arithmetic frontend. The syntax also admits a more ambitious predicate-based form `V :- body.`, but that form remains partially wired and should be viewed as reserved rather than uniformly available across all compiler entry points.

UDFs are intended to be pure: they take scalar arguments and return scalar values, and do not recurse through relations. The function registry in `crates/xlog-logic/src/function.rs` performs structural validation—duplicate definitions, unresolved calls, name conflicts, and recursive definitions without a guarded base case are rejected. Calls to arithmetic UDFs appear wherever arithmetic expressions do, including as the right-hand side of an `is`-expression. Expansion is explicit rather than universal on current `main`: selected execution paths invoke `crates/xlog-logic/src/expand.rs` to inline arithmetic UDFs into composed `Expr` trees (`Add`, `Mul`, `Conditional`, and so on), while predicate-bodied expansion remains reserved. Where this expansion path is used, UDFs participate in the same predicate-pushdown and expression-level optimization passes as built-in arithmetic.

Most classical Datalog dialects have no UDF analogue: functors are either absent or restricted to uninterpreted term construction. xlog's arithmetic UDF path closes part of this gap without breaking monotonicity. Because the integrated bodies are pure scalar expressions, a UDF call in a

recursive rule does not enlarge the set of derivable facts beyond what the surrounding Datalog rule already derives.

A UDF for score normalization:

```
pred raw_score(u32, f64).
pred norm_score(u32, f64).

func clamp01(X: f64) ->f64 =
  if X < 0.0 then 0.0
  else if X > 1.0 then 1.0
  else X.

raw_score(1, -0.3). raw_score(2, 0.7). raw_score(3, 1.4).

norm_score(Id, N) :-raw_score(Id, R), N is clamp01(R).
```

Listing 4: User-defined function: `clamp01` normalizes a raw score into the unit interval.

The `clamp01` UDF lowers to a conditional expression embedded in the `Project` computing `norm_score`'s second column; no host dispatch occurs during evaluation.

### 3.5 Modules and Imports

xlog programs decompose into modules. A module is a `.xlog` file located on the module search path, which the `xlog` CLI controls via the `-module-path` flag. The `use` directive loads a named module (e.g., `use graph.`) or selectively imports names from it (`use utils/math::{abs, clamp}.`). Module paths use `/` as a separator and mirror the on-disk directory structure. Declarations and functions carry an optional `private` modifier to hide them from importers.

Name resolution runs before type inference. The resolver in `crates/xlog-logic/src/resolver.rs` (469 lines) traverses the import graph, detects cycles, merges symbol tables, and surfaces a single logical program whose remaining compilation stages proceed as if the user had written all rules inline. Imports are transitive; attempts to reach a `private` name yield a structured `ModuleError::PrivatePredicate` diagnostic rather than a silent shadowing.

A two-module program where a top-level entry imports a graph-reachability library:

```
// library: graph_lib.xlog
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
reach(X, Z) :-reach(X, Y), edge(Y, Z).
```

Listing 5: Module `graph_lib.xlog`: predicate declarations and recursive reach rules.

```
// entry: graph_main.xlog
use graph_lib.
```

```
?-reach(1, N).
```

Listing 6: Entry `graph_main.xlog`: imports `graph_lib` and queries `reach`.

The resolver merges the two files before stratification, so the library's recursive `reach` rules participate in exactly the same SCC analysis and GPU semi-naive evaluation as if they had been written inline.

### 3.6 Aggregations and Constraints

xlog supports stratified aggregation and integrity constraints. Aggregation operators—`count`, `sum`, `min`, `max`, and `logsumexp`—appear at head positions of rules, with an explicit aggregation variable. The stratifier rejects any program in which an aggregation participates in a recursive SCC, guaranteeing a well-defined fixpoint order. Aggregations lower to `GroupBy` RIR nodes, executed on the GPU as radix-sort-and-reduce kernels in `xlog-cuda`.

Integrity constraints are headless rules: `:- body.` denotes a program-level assertion that `body` must be unsatisfiable after evaluation reaches fixpoint. A satisfied constraint is silent; a violated constraint raises a diagnostic. Constraints compose cleanly with stratification—they evaluate after the strata their body predicates inhabit.

A graph-degree computation with an integrity check that every edge source has a degree record:

```
pred edge(u32, u32).
pred degree(u32, u32).

edge(1, 2). edge(1, 3). edge(1, 4). edge(2, 3).

degree(X, count(Y)) :-edge(X, Y).

:-edge(X, _), not degree(X, _).

?-degree(X, N).
```

Listing 7: Stratified aggregation: per-source out-degree with an integrity constraint.

The `degree` rule lowers to a `GroupBy` over `edge` with `count` as the aggregator. The integrity constraint is desugared into an auxiliary rule whose body mirrors the constraint body (so the `not degree(X, _)` literal here contributes an `Anti`-join); at evaluation completion the runtime asserts the auxiliary relation is empty and raises a diagnostic otherwise. `logsumexp` is included explicitly because it is the workhorse aggregation for probabilistic circuits (Section 5), where log-space numerical stability is essential.

## 4 GPU-Native Datalog Evaluation

Section 3 describes the source language; this section describes how its Datalog subset compiles to and executes on the GPU. The deterministic evaluation engine executes standard Datalog with stratified negation and aggregation entirely on the GPU. The algorithmic approach—semi-naive fixpoint iteration over stratified programs—is well established. The contribution here is not algorithmic

novelty but engineering: every relational operator runs as a custom CUDA kernel, delta relations are maintained on-device, and no host–device transfers occur during evaluation.

### 4.1 Semi-Naive Evaluation on GPU

The executor in `crates/xlog-runtime/src/executor/` processes an `ExecutionPlan` consisting of strata ordered by the dependency analysis from Section 2. Each stratum contains one or more strongly connected components (SCCs), and the executor dispatches each SCC according to whether it is recursive or non-recursive.

**Non-recursive SCCs.** The executor evaluates each compiled rule once, passing the RIR tree to the node dispatcher which invokes the appropriate CUDA kernel for each operator (scan, join, filter, project, group-by). Results for the same head predicate are merged via the `union_gpu` kernel and deduplicated. No iteration is needed.

**Recursive SCCs.** The `execute_recursive_scc` method in `crates/xlog-runtime/src/executor/recursive.rs` implements semi-naive evaluation. The algorithm proceeds in three phases:

1. **Seeding.** All rules in the SCC are evaluated once against the current relation store to produce initial results. Per-predicate delta relations are allocated as distinct GPU-resident buffers with dedicated `RelId` identifiers (named `__delta_{pred}_{id}`). The initial delta for each predicate is computed as the set difference between the newly derived tuples and any pre-existing tuples.

2. **Iteration.** On each iteration, the executor re-evaluates rules using delta-rewritten variants. For each rule, it identifies recursive scan occurrences and rewrites each one individually to reference the corresponding delta relation, producing one evaluation variant per recursive scan site. This per-occurrence rewriting handles self-joins correctly: a rule body containing `p(X,Y), p(Y,Z)` generates two variants, each substituting the delta into exactly one scan. Variant results are unioned per head predicate. The raw delta is then differenced against the full relation (`delta_new = delta_raw - full`) and deduplicated, all on-device. A `DeltaRelationTracker` records whether any predicate produced new tuples; if none did, fixpoint has been reached.

3. **Merge and cleanup.** After each iteration, new delta tuples are merged into the full relation via `union_gpu` followed by `dedup_sorted`. When the fixpoint is reached or the configurable iteration limit (default 1,000,000, set via `RuntimeConfig.max_iterations`) is exceeded, delta relations are removed from the store and their `RelId` mappings are unregistered.

All intermediate buffers—full relations, delta relations, union results, difference results—remain GPU-resident throughout. The profiler records per-operator timing, input/output row counts, and peak GPU memory at each step, enabling the feedback loop described in Section 4.

### 4.2 Kernel Design

xlog implements eight core relational CUDA kernels, totaling 3,633 lines of device code. Each kernel is a direct CUDA implementation, not a wrapper around cuBLAS, cuSPARSE, or Thrust.

Table 1: Core relational CUDA kernels in `xlog-cuda`.

| Kernel | LOC | Purpose |
|---|---|---|
| `join.cu` | 550 | Hash join with linked-list collision chains. FNV-1a composite hashing for multi-column keys. All scalar types supported by hashing raw bytes. Includes count → scan → materialize (CSM) variants for Inner and LeftOuter (indexed and non-indexed) under env-gated dispatch. |
| `sort.cu` | 452 | 4-bit radix sort (8 passes over 32-bit keys). Stable: elements with equal digits preserve input order. Three-phase per pass: histogram, prefix sum, scatter. |
| `filter.cu` | 942 | Comparison operators (Eq, Ne, Lt, Le, Gt, Ge) for column-vs-constant and column-vs-column, with stream compaction via prefix sum. Fused compare-and-scan variants for all scalar types. |
| `groupby.cu` | 248 | Sorted-input grouped aggregation. Detects group boundaries in sorted key columns, then applies per-group reduction (Count, Sum, Min, Max, LogSumExp). |
| `dedup.cu` | 482 | Sort-based deduplication with prefix-sum compaction. Type-aware equality including IEEE 754 float handling: $-0.0$ and $+0.0$ equal, NaN equal only when bit-identical. GPU-native multi-column full-row deduplication and set difference (v0.5.5, PR #50). |
| `set_ops.cu` | 116 | Union (concat + sort + dedup) and set difference via sorted-array binary search. Columnar concat kernel copies both inputs into a pre-allocated output buffer. |
| `scan.cu` | 270 | Blelloch parallel prefix sum (inclusive). Block-level scan with shared memory, inter-block propagation via block sums. Building block for filter compaction, dedup, and group-by. |
| `pack.cu` | 581 | Multi-column key packing on device. Packs up to 4 separate column buffers into row-major byte arrays and computes FNV-1a hashes, eliminating host roundtrips for multi-column join key preparation. |

**Recorded-launch siblings.** As of v0.6.0, every operator surface in Table 1 also exposes a `*_recorded` entry point that flows through the `LaunchRecorder` preflight + commit discipline (Section 2). Recorded variants are selectable via per-operator env flags (`XLOG_USE_RECORDED_FILTERS`, `_SORT`, `_DEDUP`, `_GROUPBY`, `_HASH_JOIN`, `_CSM`) or the umbrella `XLOG_USE_RECORDED_OPS=1`; legacy paths remain the default. The CSM hash-join (`XLOG_USE_RECORDED_CSM=1`, v0.6.1) routes Inner / LeftOuter joins through count → scan → materialize phases; Semi / Anti remain on existing recorded paths.

**IEEE 754 total ordering.** The filter kernel implements a correctness-critical design choice for floating-point comparisons. Equality and inequality (Eq, Ne) use standard IEEE 754 semantics where NaN ≠ NaN. Ordered comparisons (Lt, Le, Gt, Ge) use a total ordering transformation: the `float_to_ordered_f64` device function converts an `f64` bit pattern to an `i64` such that the resulting integer order matches the IEEE 754 `totalOrder` predicate. The ordering is: $-$NaN $<$ $-$Inf $<$ negative finites $<$ $-0.0$ $<$ $+0.0$ $<$ positive finites $<$ $+$Inf $<$ $+$NaN. The transformation works by flipping all bits except the sign bit for negative values and flipping only the sign bit for non-negative values, matching Rust's `f64::total_cmp()` algorithm. An analogous `float_to_ordered_f32` function handles 32-bit floats. This ensures that Datalog programs produce deterministic, well-defined results for all floating-point inputs, including edge cases involving NaN and signed zeros.

### 4.3 Adaptive Join Planning

The optimizer in `crates/xlog-logic/src/optimizer.rs` performs cost-based query transformations using runtime statistics collected by the `StatsManager` in `crates/xlog-`

`stats/src/manager.rs`.

**Cost model.** Each plan node receives a `PlanCost` estimate with four dimensions: expected row count, CPU coordination cost, peak GPU memory, and number of host–device transfers. The scalar cost function weights these components, with transfers penalized heavily (default multiplier: 100×) to reflect PCIe latency. GPU memory cost is scaled at 0.001 per byte, making 1 GB equivalent to 1M cost units.

**Predicate pushdown.** When enabled (the default), the optimizer pushes `Filter` nodes below `Project` and `Join` nodes. Filters above joins are decomposed: predicates referencing only left-side columns are pushed into the left input, predicates referencing only right-side columns into the right input, and cross-predicates remain above the join. Filters above projections are remapped through pass-through columns where possible.

**Join ordering.** For queries involving up to `dp_threshold` relations (default: 10), the optimizer uses dynamic programming to enumerate join orderings and select the minimum-cost plan. Join cardinality estimation uses cached selectivity data from `StatsManager.estimate_join_cardinality`, which multiplies left and right cardinalities by a selectivity factor derived from prior executions. When the relation count exceeds the threshold, the optimizer falls back to a greedy heuristic to avoid exponential planning time.

**Feedback loop.** The `Executor` exposes a `stats_snapshot()` method that captures a `StatsSnapshot` containing per-relation cardinality, byte size, access heat, column-level statistics, join selectivities, and a `RelId`-to-predicate-name mapping. This snapshot can be fed back into the `StatsManager` via `merge_snapshot()` before the next compilation, closing the loop between execution and optimization. The predicate-name mapping ensures that statistics are applied correctly across compilations even when `RelId` assignments change.

### 4.4 Reversible Symbols

Datalog programs operate on strings (predicate arguments, constants), but GPU kernels operate on fixed-width numeric columns. xlog bridges this gap with a global symbol table implemented in `crates/xlog-core/src/symbol.rs`.

**Interning.** The `intern(s: &str) -> u32` function assigns sequential 32-bit IDs to unique strings. A read-write lock (`RwLock<SymbolRegistry>`) guards the bidirectional mapping: a `HashMap<String, u32>` for forward lookup and a `Vec<String>` for reverse resolution. The fast path acquires only a read lock; the slow path (new symbol) double-checks after acquiring the write lock to avoid races. Sequential allocation means IDs are dense and start at 0, which benefits GPU memory access patterns.

**Reverse resolution.** The `resolve(id: u32) -> String` function recovers the original string from an ID in $O(1)$ via the vector index. This is used at query output time to present human-readable results.

**Arrow dictionary encoding.** The `ScalarType::Symbol` variant maps to Arrow's `Dictionary(UInt32, Utf8)` type. The `to_arrow` function builds a `DictionaryArray` from a slice of symbol IDs: it collects unique IDs, resolves them to strings to form the dictionary, and remaps keys. The inverse `from_arrow` function interns all dictionary strings and maps Arrow keys back to global symbol IDs. This encoding enables zero-copy export to frameworks that consume Arrow data (cuDF, Polars, DuckDB) while preserving the compact u32 representation used internally.

**GPU representation.** On the GPU, symbol columns are stored as contiguous `u32` arrays—the same representation as any other 4-byte integer column. Join, sort, filter, and dedup kernels operate on these integer IDs without special-casing. The symbol table itself remains on the host, since string data is variable-length and only needed at ingestion and output boundaries.

# 5 Probabilistic Inference on GPU

## 5.1 Approach

Probabilistic logic programming systems such as ProbLog [12] compute the probability of queries over programs containing probabilistic facts. The standard technique is knowledge compilation: compile the ground Boolean formula derived from provenance tracking into a tractable circuit (typically a Decision-DNNF), then evaluate weighted model counts over that circuit. ProbLog follows a compile-once/evaluate-many model: the circuit structure is determined by the logical program and is independent of the probability weights, so a single compilation can serve many weight configurations during parameter learning.

xlog adopts the same compile-once/evaluate-many model but moves the entire pipeline onto the GPU. In ProbLog, provenance extraction, CNF encoding, D4 compilation, and circuit evaluation all execute on the CPU. When ProbLog is paired with a neural component (as in DeepProbLog [1]), each training iteration requires transferring circuit evaluation results from CPU to GPU for gradient computation, and pulling gradients back for weight updates. xlog eliminates this architectural split: PIR extraction, Tseitin [13] CNF encoding, D4 [7] compilation, CDCL equivalence verification, and forward/backward circuit evaluation all execute as CUDA kernels on the GPU. The compiled circuit is a GPU-resident data structure (XGCF), weight buffers and gradient buffers are GPU-resident, and no host–device transfers occur in the evaluation hot path. This design turns what would be a PCIe-bottlenecked ping-pong between CPU logic engine and GPU tensor framework into a single-device pipeline.

## 5.2 GPU Knowledge Compilation Pipeline

The probabilistic compilation pipeline transforms a Datalog program with probabilistic annotations into a GPU-resident arithmetic circuit through five stages (Figure 2). Each stage is implemented as one or more CUDA kernel launches, with intermediate data structures remaining in device memory throughout.


Figure 2: Knowledge compilation pipeline. All five stages execute as CUDA kernels on GPU-resident data; no host–device transfers occur between stages.

The five kernel files total 9,076 lines of CUDA device code. Together with the eight relational kernels from Section 4, they account for the majority of xlog's 14.4K-line CUDA codebase.

**Stage 1: PIR Extraction** (`pir.cu`, 600 LOC; host orchestration in `gpu_pir_intern.rs`). The PIR (Provenance IR) stage constructs a weighted Boolean formula from the ground program. Provenance tracking over the deterministic evaluation produces a `PirGraph` whose node types are `Lit`, `NegLit`, `And`, `Or`, and `Decision`. The GPU PIR interner performs device-side hash-consing: it uploads node batches to the GPU, computes FNV-1a hashes over node signatures (type, leaf ID, child lists), and deduplicates structurally identical subformulas using a GPU hash table with atomic insert operations. This deduplication is critical because grounding can produce exponentially many redundant subformulas; interning on device avoids materializing the full uninterned graph in host memory. The interner outputs a `GpuPirGraph` containing device-resident arrays for node types, child offsets, child indices, leaf IDs, and decision variable metadata.

**Stage 2: CNF Encoding** (`cnf.cu`, 623 LOC; host orchestration in `gpu_cnf.rs`). The CNF encoder transforms the PIR graph into a conjunctive normal form formula using the Tseitin transformation, executed entirely on GPU. The kernel pipeline proceeds in six phases: (1) BFS reachability from query roots to identify live nodes; (2) classification and prefix-sum counting of leaf variables (probabilistic facts) and choice variables (annotated disjunction selectors); (3) variable assignment via exclusive scans that produce compact, gap-free variable IDs; (4) Tseitin clause counting per node (AND nodes produce implication clauses, OR nodes produce reverse-implication clauses, Decision nodes produce if-then-else clauses); (5) clause and literal offset computation via parallel prefix sum; (6) clause emission into a GPU-resident CSR (compressed sparse row) representation. The output is a `GpuCnfEncoding` containing the CSR clause structure (`GpuCnf`) and variable mapping tables that link PIR node IDs to CNF variable IDs. The encoder also computes a `decision_var_limit` that separates semantically meaningful variables (leaf and choice variables eligible for branching) from internal Tseitin variables used only for unit propagation.

**Stage 3: D4 Compilation** (`d4.cu`, 2,953 LOC; host orchestration in `gpu_d4/mod.rs` and `gpu_d4/build.rs`). The D4 compiler transforms the GPU-resident CNF into a Decision-DNNF circuit stored in the XGCF format. This is the most complex stage, implementing a GPU-native variant of the D4 algorithm [7]. The compiler uses a hybrid BFS/DFS strategy controlled by `GpuCompileConfig`: it performs BFS expansion to a configurable frontier depth to expose parallelism, then hands each frontier item to a per-block DFS worker. Each DFS worker performs component detection (identifying independent sub-CNFs via variable connectivity), decision variable selection (using VSIDS-like activity scores [14]), and recursive decomposition. The compiler emits circuit nodes—`Const0`, `Const1`, `Lit`, `And`, `Or`, `Decision`—into a flat device-resident array. After initial compilation, a GPU smoothing pass ensures that all branches of each OR/Decision node mention the same set of random variables, which is required for correct weighted model counting. The smoothed circuit is levelized: nodes are sorted into topological levels via a BFS from leaves, with `level_offsets` and `level_nodes` arrays enabling level-parallel evaluation. The output is a `GpuXgcf` structure ready for forward and backward passes. Configuration parameters include `max_frontier_items` (hard cap on BFS work items), `max_depth` (defensive recursion limit), and `smooth_node_cap`/`smooth_edge_cap` (bounds on smoothing pass output).

**Stage 4: CDCL Verification** (`sat.cu`, 3,268 LOC; host orchestration in `gpu_cdcl.rs` and `compilation/validation.rs`). After D4 produces a circuit $C$ from the CNF formula $\varphi$, the pipeline must verify that the circuit is semantically equivalent to the original formula: $\varphi \equiv C$. This verification is not a heuristic check or a sampling-based confidence test—it is a complete formal proof. The verifier constructs two equivalence queries on the GPU: $q_1 = \varphi \wedge \neg C$ and $q_2 = C \wedge \neg\varphi$. If both queries are unsatisfiable, then $\varphi$ and $C$ agree on every possible variable assignment, establishing

logical equivalence. Each query is solved by the GPU CDCL (Conflict-Driven Clause Learning) solver, which implements a full SAT solver as a single-block CUDA kernel (256 threads) with watched-literal propagation, 1-UIP conflict analysis, VSIDS decision heuristic, deterministic restarts, and periodic clause-database reduction. The solver operates entirely on device-resident data: variable assignments, decision levels, reason clauses, activity scores, the trail, watched-literal lists, learned clause arena, and resolution proof trace are all GPU-resident buffers. For UNSAT results, the kernel produces a resolution proof trace that is verified on-device by a separate proof-checking kernel (`sat_proof_mark_needed` + `sat_proof_check` + `sat_assert_ok`), ensuring that the UNSAT claim is backed by a machine-checkable certificate. The `solve_expect_unsat` family of methods enforce the UNSAT expectation on GPU without any device-to-host status reads—if the solver returns SAT when UNSAT was expected, the kernel triggers a device-side trap. Pre-allocated `GpuCdclWorkspace` structures amortize the allocation of the solver's 30+ device buffers across multiple solves.

**Stage 5: Circuit Evaluation** (`circuit.cu`, 1,632 LOC; host orchestration in `gpu.rs`). The compiled and verified XGCF circuit supports two evaluation modes: a forward pass for weighted model counting and a backward pass for gradient computation. The forward pass iterates over topological levels from leaves to root, launching one `xgcf_forward_level` kernel per level. Each kernel thread processes one node: `Const0` nodes receive log-value $-\infty$, `Const1` nodes receive log-value 0, `Lit` nodes look up the log-weight of their variable (positive or negative literal), `And` nodes sum their children's log-values, `Or` nodes compute log-sum-exp over their children's log-values, and `Decision` nodes combine their true-branch and false-branch values weighted by the decision variable's log-weights. All arithmetic is in log-space to avoid floating-point underflow. The root node's value after the forward pass is $\log Z$, the log partition function (or $\log Z \mid \text{evidence}$ when evidence constraints are applied). The backward pass iterates in reverse topological order, launching three kernels per level: `xgcf_backward_level_propagate` distributes adjoints from parent nodes to children (additive for OR/Decision, pass-through for AND), `xgcf_backward_level_decision_grad` accumulates gradients for decision variables, and `xgcf_backward_level_lit_grad` accumulates gradients into per-variable `grad_true` and `grad_false` buffers. These gradient buffers are GPU-resident and can be consumed directly by PyTorch's autograd graph via DLPack (Section 6).

### 5.3 Circuit Caching

D4 compilation is the most expensive stage in the pipeline. On the MNIST addition benchmark (`01_minimal`, 512 training examples), a cold-start compilation takes approximately 75 seconds. Since this cost is incurred at the beginning of each training epoch in a naive implementation, it can dominate total training time.

The key insight enabling circuit caching is that the XGCF circuit structure depends on the logic program—specifically, on the propositional structure of the grounded rules and the evidence pattern—but not on the numerical values of the probability weights. When neural network parameters change between training iterations, the weights assigned to probabilistic facts change, but the Boolean formula and its compiled Decision-DNNF structure remain identical. This means the compiled XGCF can be cached and reused across all training iterations, with only the weight buffers updated before each forward pass.

The `GpuCircuitCache` in `crates/xlog-prob/src/compilation/gpu_cache.rs` implements this caching. On the first evaluation, the full pipeline runs (PIR extraction, CNF encoding, D4 compilation, CDCL verification), and the resulting `GpuXgcf` is stored in the cache keyed by a

structural hash of the PIR graph and evidence configuration. On subsequent evaluations, the cache returns the existing circuit, and only the weight-upload and forward/backward passes execute. The cache also stores the CNF variable tables (`GpuCnfVarTables`) needed to map between PIR leaf IDs and CNF variable IDs for weight construction.

Measured impact: on the MNIST addition benchmark (3 epochs, 512 training examples, 3 seeds), cache-enabled training completes in a mean of 88.89 seconds (std: 3.48 s), compared to 242.90 seconds (std: 8.19 s) with caching disabled. This yields a 2.74× end-to-end speedup (95% CI: [2.29, 3.18]). The speedup is large because D4 compilation cost is amortized across all epochs: in the latest checked-in post-fix rerun on `main`, the first epoch averages 71.654 s while warm epochs average 0.248 s once the cached circuit and batched evaluation path are active (Section 8).

### 5.4 Monte Carlo Sampling

For programs where exact inference via knowledge compilation is infeasible—either because the ground formula is too large for D4 compilation within the available GPU memory, or because the user prefers approximate results with bounded computation—xlog provides a GPU-parallel Monte Carlo sampling engine implemented in `crates/xlog-prob/src/mc/`.

The MC engine supports two sampling methods, selectable via `McSamplingMethod`:

- **Rejection sampling.** The sampler draws values for all probabilistic facts from their prior distributions on the GPU, then evaluates the deterministic Datalog program in each sampled world. Worlds where the evidence is not satisfied are discarded. Query probabilities are estimated as the fraction of accepted worlds in which each query atom is derived. This method is unbiased but can be inefficient when the evidence has low prior probability, as most samples are rejected.
- **Evidence clamping.** When all evidence atoms correspond to Bernoulli probabilistic facts, the sampler can force those facts to their observed values, guaranteeing that every sample satisfies the evidence. This eliminates rejection waste entirely: the `McCountStrategy` switches to `QueriesOnly` mode, skipping evidence-side buffer allocations and truth-kernel evidence checks. Evidence clamping is auto-selected when the `EvidenceForcing` analysis determines that all evidence is forceable (i.e., each evidence atom maps to a single probabilistic fact with no intermediate rules).

Both methods report confidence intervals for estimated probabilities. The sampling loop is structured to minimize per-sample overhead: probabilistic fact values are sampled in bulk on the GPU via the `sampling` submodule, the relation store is reset using a targeted `McSampleResetPlan` that preserves pure deterministic relations and clears only dynamic relations (avoiding a full store clone per sample), and per-sample pointer buffers are pre-allocated outside the sample loop. The current repository contains these hot-loop optimizations in code, but does not ship a standalone archived MC timing report with stable phase-by-phase numbers, so this paper limits itself to the implementation strategy rather than quoting unsupported per-benchmark deltas.

### 5.5 Well-Founded Semantics

Standard stratified negation requires that no recursive cycle passes through a negated literal. Programs that violate this constraint—such as the classic `p :- not q. q :- not p.`—have no

two-valued stable model and are rejected by most Datalog engines. xlog handles such non-monotone programs via Well-Founded Semantics (WFS), implemented in `crates/xlog-prob/src/wfs.rs`.

WFS assigns three-valued truth to atoms: true, false, or undefined. The algorithm computes an alternating fixed point: (1) find the greatest unfounded set (atoms that cannot be supported by any rule), mark them false; (2) derive all consequences of the current knowledge, mark them true; (3) repeat until stable. Atoms that remain neither true nor false are classified as undefined. For probabilistic programs, true atoms receive normal probability and gradient computation, while false and undefined atoms are assigned probability zero with zero gradients—matching ProbLog's conservative treatment of non-stratified programs.

WFS is invoked automatically during provenance extraction when a non-monotone SCC is detected. The Monte Carlo engine also handles non-monotone programs: when cycles are detected during per-sample fixpoint evaluation, the MC engine uses the intersection of all states in the cycle (skeptical, invariant tuples only) as the interpretation, providing a deterministic semantics that avoids parity dependence on iteration count. An example program exercising this path is `examples/prob/04-nonmonotone-mc.xlog`, which defines the mutual-negation cycle `p :- flip. q :- not p. p :- not q.` and queries both `p` and `flip` under Monte Carlo inference.

# 6 Neural-Symbolic Bridge

The previous sections describe xlog as a purely symbolic system: Datalog evaluation on the GPU (Section 4), probabilistic inference through knowledge compilation (Section 5). This section introduces the neural-symbolic bridge—the machinery that connects PyTorch neural networks to the probabilistic logic engine, enabling end-to-end differentiable learning where neural perception and logical reasoning are jointly trained.

## 6.1 Neural Predicates

In xlog, a neural network *is* a predicate. The `nn/4` declaration embeds a network directly into the Datalog program:

```
nn(network_name, [input_vars], output_var, [output_labels]) ::predicate(args).
```

Listing 8: Neural predicate declaration syntax (`nn/4`).

This is not a foreign-function call or an escape hatch. The declaration states that evaluating predicate `predicate(args)` requires a forward pass through the named network. The network's softmax output over the label set becomes a distribution over probabilistic facts, which then participates in the standard knowledge compilation pipeline described in Section 5.

On the Python side, networks are registered via the `pyxlog` API. The `CompiledProgram.register_network()` method accepts a PyTorch `nn.Module`, an optimizer, and optional configuration parameters:

```
program.register_network("mnist_net", net, optimizer)
```

Listing 9: Registering a PyTorch module against a neural predicate via `pyxlog`.

Under the hood, the Rust-side `NetworkRegistry` (in `crates/xlog-neural/src/registry.rs`) manages a collection of `NetworkHandle` instances, each holding a reference to the PyTorch module, optimizer, and optional learning rate scheduler. The `NetworkConfig` struct exposes builder-pattern configuration for batching (grouping multiple queries into a single forward pass), top-k sampling (restricting the output space when most classes have near-zero probability), deterministic mode (argmax instead of probabilistic sampling), and output caching (avoiding redundant forward passes for repeated inputs, up to a configurable cache size of 10,000 entries by default).

The data flow through a neural predicate proceeds in two directions. **Forward:** when the logic engine encounters an `nn/4`-backed predicate during evaluation, it invokes the registered PyTorch module. The network's softmax output vector is decomposed into weighted probabilistic facts—one per output label—which are fed into the PIR/CNF/D4/XGCF compilation pipeline. Because the circuit structure depends on the logic program and not on the weight values, the compiled XGCF template is cached (Section 5) and only the leaf weights are updated on subsequent iterations. **Backward:** after the XGCF circuit computes the forward log-probability and backward gradients via level-parallel CUDA kernels, the per-leaf gradient buffers are exported as DLPack tensors and injected into PyTorch's autograd graph. The optimizer then updates the network parameters as usual. The Python GIL is released during GPU-resident work using `py.allow_threads()`, ensuring that CUDA kernel execution and GPU memory operations do not block the Python interpreter.

### 6.2 End-to-End Training Loop

To make the integration concrete, we walk through the MNIST addition example—the canonical DeepProbLog benchmark. The task: given two MNIST digit images, predict their sum. The network never sees individual digit labels; it learns to classify digits purely from supervision on sums.

**The logic program.** The complete xlog source (`examples/neural/01_minimal/minimal.xlog`) is:

```
// Neural predicate declaration
// nn(network_name, [input_vars], output_var, [output_labels]) ::predicate(args).
//
// This declares that:
// - 'mnist_net' is the name of the neural network
// - [X] means the network takes one input (image at index X)
// - Y is the output variable (the classified digit)
// - [0,1,2,3,4,5,6,7,8,9] are the possible output labels (digit classes)
// - digit(X, Y) is the resulting predicate
//
// The network's softmax output becomes probabilistic facts:
// P(digit(0, 0)) = network_output[0]
// P(digit(0, 1)) = network_output[1]
// ...
// P(digit(0, 9)) = network_output[9]
nn(mnist_net, [X], Y, [0,1,2,3,4,5,6,7,8,9]) ::digit(X, Y).

// Addition rule
// The sum of two digits at indices X and Y is Z.
//
// This uses probabilistic inference:
```

```
// P(addition(0, 1, 7)) = sum over all (d1, d2) where d1 + d2 = 7 of:
// P(digit(0, d1)) * P(digit(1, d2))
//
// For example, with d1=3, d2=4:
// P(addition(0, 1, 7)) includes P(digit(0, 3)) * P(digit(1, 4))
addition(X, Y, Z) :-digit(X, D1), digit(Y, D2), Z is D1 + D2.
```

Listing 10: MNIST addition program: a CNN-backed digit predicate plus an addition rule.

Two lines of logic define the entire reasoning structure. The `nn/4` declaration connects the `mnist_net` CNN to a `digit/2` predicate. The addition rule computes all consistent decompositions of a sum through probabilistic marginalization.

**The training script.** The Python driver (`examples/neural/01_minimal/train.py`) compiles the program, registers the network, and runs training through the `pyxlog.train_model_tensor` API:

```
program = pyxlog.Program.compile("""
    nn(mnist_net, [X], Y, [0,1,2,3,4,5,6,7,8,9]) ::digit(X, Y).
    addition(X, Y, Z) :-digit(X, D1), digit(Y, D2), Z is D1 + D2.
""")

program.register_network("mnist_net", net, optimizer)
program.add_tensor_source("train", train_images)

queries = generate_queries(n_pairs, train_labels)

history = pyxlog.train_model_tensor(
    program,
    queries,
    epochs=args.epochs,
    batch_size=args.batch_size,
    log_iter=args.log_iter,
)
```

Listing 11: Python driver for MNIST addition: compile, register network, train.

Each query has the form `addition(i, j, expected_sum)`. The training objective minimizes the negative log-probability of the query under the compiled circuit: $-\log P(\texttt{addition}(i, j, s))$. The `train_model_tensor` function orchestrates the full loop: batching queries that share the same circuit template, running forward and backward passes over the cached XGCF circuit, and propagating gradients back to the PyTorch optimizer.

**Training loop architecture.** Figure 3 shows one training iteration.

The critical performance property is that the circuit compilation (D4) happens once, during the first forward pass. All subsequent iterations reuse the cached XGCF structure, executing only the weight update and the level-parallel forward/backward kernels. This is the mechanism behind the 2.74× training speedup reported in Section 5.

**Results.** On the MNIST addition benchmark (512 training images, 256 addition pairs, batch size 64, learning rate $10^{-3}$, CUDA device), the latest checked-in post-fix rerun on `main` reports the summary in Table 2 over 3 seeds:

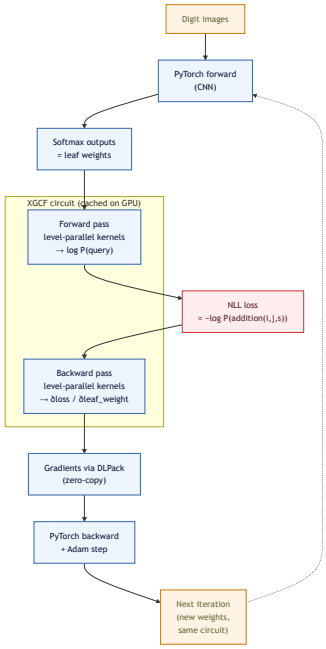


Figure 3: One iteration of neural-symbolic training on MNIST addition. The XGCF circuit is compiled once (first iteration) and reused thereafter; only the leaf weights change across iterations. Gradients flow back through DLPack capsules into PyTorch's autograd graph.

Table 2: MNIST addition training summary (`01_minimal`, post-fix rerun, 3 seeds).

| **Metric** | **Value** | **Artifact** | **Notes** |
|---|---|---|---|
| Mean first epoch | 71.654 s | Track B comparison | Cold start with D4 compile/verify |
| Mean steady epoch | 0.248 s | Track B comparison | Warm batched path after cache reuse |
| Mean total train | 72.645 s | Track B comparison | 5 epochs, 3 seeds |
| Mean final loss | 2.62953 | Track B comparison | Final training loss over the 3 recorded seeds |

These measurements come from the post-fix Track B comparison artifact at `examples/neural/results/track_b/20260218T145032Z_track_b/comparisons/mnist_xlog_vs_scallop.json`, which exercises the full xlog pipeline—compilation, circuit caching, GPU-resident forward/backward, and batched query evaluation—through `pyxlog.train_model_tensor`. The loss reduction across the recorded runs confirms that gradients flow from circuit evaluation through the logic program back to the CNN parameters, and the timing split matches the cold-start vs. warm-epoch behavior reported in Section 8.

### 6.3 Term Embeddings (v0.5.0)

Version 0.5.0 introduces a term embedding API that attaches dense vector representations to Datalog symbols. Where `nn/4` neural predicates map perception (images, text) to discrete logical facts, term embeddings go in the opposite direction: they give logical entities a continuous representation that can participate in neural computation.

The Python API provides two methods on `CompiledProgram`:

- **`register_embedding(name, module_or_tensor, trainable=True)`** accepts either a PyTorch `nn.Embedding` module or a raw 2D `torch.Tensor`. For `nn.Embedding`, the weight matrix shape [vocab_size, dim] is read from the module. For raw tensors, `trainable` must be `False` (frozen embeddings). The Rust-side `EmbeddingHandle` (in `crates/xlog-neural/src/handle.rs`) stores the module reference, dimensionality, vocabulary size, and trainability flag. Embeddings are registered in the same `NetworkRegistry` as neural predicates, keyed by name.
- **`forward_embedding(name, ids)`** looks up embedding vectors by integer IDs, returning a batched `torch.Tensor` with shape [len(ids), dim]. For trainable `nn.Embedding` modules, the returned tensor carries autograd history, so gradients flow through the embedding lookup. For frozen tensors, the result has `requires_grad=False`. Internally, the method determines the device from the embedding weight tensor and creates the ID tensor on the same device to avoid CPU/CUDA mismatches.

This API enables entity representation learning within the logic framework. For example, a knowledge graph completion task can attach learned embeddings to entity symbols, combine them with relational predicates in xlog rules, and jointly train the embeddings alongside neural perception networks. The embeddings are the only gradient-carrying embedding API as of v0.6.1; future releases may integrate embedding lookups directly into the circuit evaluation path.

### 6.4 Differentiable ILP (Beta)

> **Note:** The differentiable ILP subsystem is in **beta** as of v0.6.1. The API surface, promotion pipeline, and artifact format may change in future releases. The schema version is `beta-v2`.

xlog includes a differentiable inductive logic programming (dILP) trainer [6] that learns first-order rules from examples. Unlike the neural predicate path (which trains network parameters given fixed rules), dILP searches the space of candidate rules, using gradient-based optimization to assign soft probabilities to candidates and hard selection to commit the best rule.

**Sparse GPU mask API.** A key scalability challenge in dILP is the candidate mask computation. For a program with $R$ relations of arity $A$, the naive candidate space is $O(R^A)$, which can easily reach millions of candidates. Materializing a dense $N \times N \times N$ mask on the Python side is prohibitive. The `CompiledIlpProgram` class (in `crates/pyxlog/src/ilp.rs`) provides three sparse mask methods of increasing strictness:

- `set_rule_mask_sparse(name, candidate_ids, soft_probs_dlpack, budget)`—accepts a DLPack GPU tensor of soft probabilities, performs deterministic top-k selection on the Rust side, and stores a sparse `IlpMask` without dense materialization.
- `set_rule_mask_sparse_selected(name, selected_candidate_ids, selected_soft_probs_dlpack)`—the caller preselects candidate IDs, avoiding the full-vector probability download entirely.
- `set_rule_mask_sparse_selected_device(name, ...)`—the strictest path where selected candidate IDs remain as a device tensor, with Rust resolving them against a fixed candidate order uploaded via `set_candidate_map()`.

The GPU loss/gradient computation (`compute_ilp_loss_grad_gpu`) builds a sparse COO structure from the per-entry membership masks, converts it to CSR via device-side sort, and launches forward (credit gather + NLL loss) and backward (gradient scatter) CUDA kernels. The entire credit/loss path is GPU-resident with zero host transfers when using the strict sparse device mask.

**Six-gate promotion pipeline.** When training converges and a candidate rule is selected, xlog does not blindly commit it to the program. Instead, the rule passes through a six-gate promotion pipeline:

1. **Convergence gate**—verifies that training actually converged (loss plateau, discreteness threshold).
2. **Novel-rate audit**—checks the fraction of new derivations the rule produces; a configurable `max_novel_rate` threshold catches rules that overgenerate.
3. **Regression check**—ensures the new rule does not invalidate existing derivations for protected relations.
4. **Holdout F1 gate**—evaluates the candidate on held-out examples using k-fold cross-validation (default 5 folds, threshold 0.95 F1).
5. **Ambiguity scan**—detects when multiple candidates have similar soft probabilities, optionally reporting `ambiguous_alternatives` for manual review.
6. **Typed schema validation**—when `typed_schema_required=True`, verifies that the candidate rule's variable bindings respect declared type annotations on relations.

A rule that fails any gate receives a `PromotionStatus` of `GATE_FAILED` or `MANUAL_REVIEW_REQUIRED`, with per-gate `GateResult` details. Only rules that pass all six gates are committed.

**Hard-negative mining.** The `TrainConfig.max_mined_negatives` parameter enables hard-negative mining: the trainer identifies examples that the current candidate rule incorrectly covers and adds them as negative examples for subsequent training rounds. This improves precision for rules operating in domains with many near-miss patterns.

**Artifact persistence.** Learned rules are serialized as `LearnedArtifact` objects containing the candidate map, final logits, soft probabilities, hard selection, discovered rule text, full `TrainConfig` snapshot, and training telemetry. The `save()` method writes JSON with SHA-256 hashes computed over both the candidate map and the configuration snapshot, enabling integrity verification on `load(path, verify_hash=True)`. The schema version (`beta-v2`) is validated on load to prevent silent format incompatibilities. For strict GPU-native training paths where logits and probabilities remain device-resident, a `StrictLearnedArtifact` variant defers host materialization until `export_compat_artifact()` is explicitly called, avoiding unnecessary device-to-host transfers during automated pipeline runs.

# 7 Interoperability

A GPU-native logic engine is only as useful as its ability to exchange data with the frameworks researchers already use. xlog exposes GPU-resident relations, probabilities, and gradients through four standard interfaces—DLPack, Arrow, Python bindings, and PyTorch autograd—so that results flow into downstream tools without device-to-host copies or format conversion.

## 7.1 DLPack: Zero-Copy GPU Tensor Sharing

The `xlog-cuda` crate implements the DLPack [8] protocol in `crates/xlog-cuda/src/dlpack.rs`, providing bidirectional zero-copy exchange of GPU buffers with any framework that speaks DLPack—including PyTorch, JAX, CuPy, and cuDF. Export is handled by `DlpackTable`, which wraps a reference-counted `CudaBuffer` and produces per-column `DLManagedTensor` handles with a registered deleter callback; the underlying GPU memory remains shared and is freed only when all consumers release their handles. Import is handled by `CudaKernelProvider::from_dlpack_tensors`, which accepts a vector of `DLManagedTensor` pointers—one per column—validates device affinity, dtype, contiguity, and alignment, and assembles them into a `CudaBuffer` without any device-to-host copy. A schema-checked variant (`from_dlpack_tensors_with_schema`) adds compile-time type verification against an expected `Schema`, rejecting mismatches before evaluation begins. All xlog scalar types—`U32`, `U64`, `I32`, `I64`, `F32`, `F64`, `Bool`, and `Symbol`—have well-defined DLPack dtype mappings. The protocol is surfaced to Python as PyCapsule objects with the `"dltensor"` name; consumers call `torch.from_dlpack(capsule)` to obtain a live CUDA tensor backed by the same device memory that xlog wrote.

## 7.2 Arrow IPC: Columnar Export with Dictionary Encoding

For integration with columnar analytics tools—Pandas, Polars, DuckDB—xlog exports relations through the Arrow [9] C Data Interface. The `crates/xlog-cuda/src/arrow_device.rs` module defines `ArrowDeviceArray`, a `repr(C)` struct that pairs Arrow FFI array and schema pointers with a CUDA device type and ordinal, following the Arrow C Device specification. The Python API exposes `export_arrow_device` and `import_arrow_device` for zero-copy round-tripping between xlog's GPU

buffers and Arrow-compatible consumers. Symbol-typed columns receive special treatment: `xlog-core`'s `symbol::to_arrow` function (`crates/xlog-core/src/symbol.rs`) converts interned symbol IDs into Arrow `DictionaryArray<UInt32Type>` with a `StringArray` dictionary, preserving the human-readable string mapping while keeping the integer representation compact. The inverse path, `symbol::from_arrow`, re-interns dictionary values back into xlog's global symbol table. This dictionary encoding means that a Polars or DuckDB consumer can read symbolic relation columns as native string dictionaries without a separate lookup step.

### 7.3 Python Bindings

The `pyxlog` crate provides Python bindings built with PyO3 [11]. The public API—documented in `crates/pyxlog/python/pyxlog/_native.pyi`—exposes three compilation entry points (`LogicProgram.compile`, `Program.compile`, `IlpProgramFactory.compile`), each returning a compiled program object with evaluation, training, and result-extraction methods. Type stubs (`.pyi` files) provide full IDE auto-completion and static type checking. All tensor-valued attributes—`EvalResult.prob`, `McDeviceEvalResult.query_counts`, `IlpTaggedCreditDeviceResult` fields—are returned as DLPack capsules, keeping the data on the GPU and letting the caller choose how to materialize it. Long-running GPU operations release the Python GIL via `py.allow_threads()` before entering CUDA kernel dispatch—circuit evaluation, ILP fixpoint iteration, and the neural backward pass all execute without holding the GIL, allowing Python data-loader threads and asyncio tasks to run concurrently with GPU computation.

### 7.4 PyTorch Autograd Integration

Neural-symbolic training requires gradients to flow from logical inference back into neural network parameters. xlog achieves this by constructing loss tensors that participate directly in PyTorch's autograd graph. The `forward_backward_tensor` method in `crates/pyxlog/src/neural.rs` evaluates the compiled circuit on the GPU—with the GIL released—then exports the scalar NLL loss as a DLPack capsule and imports it into PyTorch via `torch.from_dlpack`. Because the neural network's forward pass produced grad-enabled output tensors that fed into the circuit, the resulting loss tensor remains connected to the autograd graph. The batched path (`forward_backward_batch_complex_tensor`) goes further: it stacks per-network inputs into a single batched forward pass, evaluates multiple queries against the shared circuit, accumulates per-query losses, and then calls `torch.autograd.backward(outputs, grad_tensors)` for a single backward pass through the entire computation graph. This design means that xlog's probabilistic circuit acts as a differentiable function inside PyTorch—gradients flow from the logic layer through DLPack tensors back to `nn.Module` parameters, with no custom autograd `Function` subclass required.

## 8 Evaluation

This section reports a mix of artifact-backed measurements and benchmark acceptance targets for xlog's deterministic, probabilistic, and neural-symbolic subsystems. Every quantitative claim is tied either to a checked-in artifact or to the benchmark contract in `docs/BENCHMARKS.md`; the two are distinguished explicitly below. These are single-system measurements on development hardware, not controlled head-to-head comparisons against other engines.

## 8.1 Methodology

**Hardware.** All measurements were collected on an NVIDIA RTX PRO 3000 Blackwell Generation Laptop GPU (12 GB VRAM, SM120, compute capability 12.0, driver 591.59) (`docs/BENCHMARKS.md`).

**Build configuration.** Rust crates are compiled in `-release` mode. CUDA kernel modules are staged cubin-first with portable PTX fallback and explicit JIT warmup. Python components use `pyxlog` with CUDA-enabled PyTorch.

**Statistical protocol.** The benchmark harness uses Criterion.rs with the settings in Table 3 (`docs/BENCHMARKS.md`).

Table 3: Criterion.rs benchmark settings.

| Setting | Value |
|---|---|
| Sample size | 10–100 runs (benchmark-dependent) |
| Warm-up | 3 iterations (PTX JIT, memory pool init) |
| Significance level | 0.10 |
| Noise threshold | 0.05 (ignore <5% variance) |
| Seeding | Deterministic LCG (no system entropy) |

Artifact-backed values are quoted directly from checked-in JSON/Markdown evidence. Tables sourced from `docs/BENCHMARKS.md` are benchmark targets for the development machine rather than archived run logs. GPU measurements include explicit warm-up to amortize kernel-module loading and CUDA context establishment.

## 8.2 Absolute Performance

### 8.2.1 Deterministic Subsystem

The GPU logic engine (`xlog-gpu`) targets throughput on three core operations: transitive closure (recursive fixpoint), hash join, and aggregation. Table 4 reproduces the current benchmark contract from `docs/BENCHMARKS.md`; for deterministic execution the repository publishes target envelopes, not archived Criterion result tables.

Table 4: Deterministic benchmark targets from `docs/BENCHMARKS.md`.

| Operation | Configuration | Benchmark Contract |
|---|---|---|
| Transitive closure | 100K random edges (sparse) | >1M rows/sec |
| Transitive closure | 1M random edges (medium) | >5M rows/sec |
| Transitive closure | $K_{500,500}$ bipartite (dense) | >10M rows/sec |
| Hash join | 100K × 100K | >50M rows/sec |
| Hash join | 1M × 100K | >100M rows/sec |
| Hash join | High selectivity | >20M rows/sec |
| Aggregation | 1M rows, 100K groups | Tracked in groups/sec; no fixed public threshold yet |

These targets represent the current performance contract on the development GPU. Actual measured throughput depends on graph structure, key distribution, and output cardinality. The

semi-naive fixpoint iteration benchmark (`crates/xlog-gpu/benches/logic_bench.rs`) exercises the full recursive evaluation pipeline including delta maintenance and deduplication.

### 8.2.2 Probabilistic Subsystem

The probabilistic engine (`xlog-prob`) supports two inference modes: exact inference via Decision-DNNF knowledge compilation and GPU-accelerated Monte Carlo sampling.

Table 5: Exact inference targets (`docs/BENCHMARKS.md`).

| **Configuration** | **Target** | **Notes** |
|---|---|---|
| 20-variable probabilistic path | <100 ms | Small d-DNNF circuit |
| 50-variable Bayesian network | <500 ms | Medium circuit complexity |
| With gradient computation | <2× base time | Backward pass overhead |

Table 6: Monte Carlo inference targets (`docs/BENCHMARKS.md`).

| **Configuration** | **Target** | **Notes** |
|---|---|---|
| 100K samples, 100 variables | >10M worlds/sec | Throughput mode |
| 10K samples, 500 variables | >5M worlds/sec | Complexity mode |

**MC evidence status.** The implementation clearly contains the relevant hot-loop optimizations—targeted reset planning, bulk GPU sampling, and pointer-buffer reuse—but the repository does not currently ship a standalone archived MC timing report with stable phase-by-phase numbers. Accordingly, this section treats MC performance as target-level evidence (Table 6) rather than quoting unsupported absolute timings.

### 8.2.3 Neural-Symbolic Subsystem

Neural-symbolic training is measured on the `01_minimal` benchmark (MNIST single-digit addition, 512 training images, 5 epochs, batch size 64). The timing summary below uses the latest checked-in post-fix comparison artifact on current `main`; the cache-ablation study remains a separate 3-epoch measurement of the compilation-cache effect.

Table 7: Training performance summary.

| **Metric** | **Value** | **Source** |
|---|---|---|
| PTX JIT cold start | 0.02 s | Cubin-first module loading target from `docs/BENCHMARKS.md` |
| First epoch (cold) | 71.654 s | Track B comparison, mean over 3 seeds |
| Steady-state epoch (warm) | 0.248 s | Track B comparison, mean epochs 2–5 |
| Total training (5 epochs) | 72.645 s | Track B comparison, mean over 3 seeds |
| Per-query cost | 0.97 ms | Track B comparison, mean forward+backward query cost |
| Circuit cache speedup | 2.74× | Cache ablation artifact, 95% CI: [2.29, 3.18] |

The cache ablation study compared circuit-cached vs. uncached training over 3 seeds and 3 epochs each. Mean training time dropped from 242.9 s (uncached) to 88.9 s (cached), yielding the

2.74× speedup reported in Section 5 (`examples/neural/results/evidence/cache_ablation_20260218.json`).

**DeepProbLog baseline.** The checked-in DeepProbLog comparison artifact is partial and protocol-mismatched, so it should be read as a qualitative baseline note rather than a clean head-to-head measurement. In `examples/neural/results/track_a/20260216T145409Z_track_a_dev/comparisons/mnist_vs_deepproblog.json`, the strict full-train DeepProbLog configuration (2 epochs, batch size 2048) reached held-out addition accuracy of 23.24% after 2:31:07, while a smaller limited run was manually aborted after 60 minutes. The same artifact reports XLOG training loss rather than held-out accuracy and marks one XLOG seed incomplete, so direct quantitative comparison remains premature.

**GA-based ILP.** The genetic algorithm reliability gate (`test_ga_reliability_50`) achieved a 3.25× wall-clock reduction from 1447 s to 445.30 s by reducing redundant attempts, with identical statistical quality: 200/200 success rate and Clopper–Pearson lower-95 CI = 0.981725 (`docs/evidence/2026-03-05-ga-runtime-closure/sweep-results.md`).

### 8.3 Qualitative Comparison

Table 8 positions xlog relative to existing systems across key capability dimensions. Each system addresses a subset of the design space; xlog's contribution is unifying these capabilities within a single GPU-resident runtime.

Table 8: Capability comparison across probabilistic and GPU logic systems.

| **Dimension** | **DeepProbLog** | **ProbLog2** | **GPUlog** | **xlog** |
|---|---|---|---|---|
| Symbolic execution | CPU (SWI-Prolog) | CPU (tabling) | GPU (HISA) | GPU (semi-naive fixpoint) |
| Probabilistic inference | Exact (PySDD, CPU) | Exact + approx (CPU) | None | Exact (d-DNNF, GPU) + MC (GPU) |
| Knowledge compilation | PySDD (CPU) | d-DNNF via c2d/D4 (CPU) | None | D4 compile + GPU circuit eval |
| Neural integration | PyTorch + Prolog bridge | None | None | Zero-copy DLPack, fused backward |
| Zero-copy ML interop | No (CPU–GPU transfers) | No | No | Yes (DLPack, device-resident tensors) |
| Differentiable ILP | Via TP-operator (CPU) | No | No | GPU-resident credit/loss path (beta) |

**Published GPU Datalog results.** GPUlog [3] reports up to 45× speedup over CPU baselines using HISA (Hash-Indexed Sorted Array) indexing with lock-free deduplication. VFLog [4] demonstrates up to 200× gains over CPU column engines with a column-oriented GPU Datalog runtime. Both systems are restricted to deterministic evaluation; neither supports probabilistic inference, knowledge compilation, or neural network integration. xlog draws on the same insight—that Datalog's relational algebra maps naturally to GPU parallelism—but extends the execution model through the probabilistic and neural-symbolic layers described in Sections 5 and 6.

**Honest framing.** The comparison above is qualitative. xlog has not been benchmarked head-to-head against GPUlog or VFLog on identical workloads. The deterministic throughput targets in Table 4 represent xlog's own benchmark contract, not comparative claims. The DeepProbLog comparison is limited to the `01_minimal` task with differing dataset sizes, metrics, and partially completed archived runs, as noted above.

### 8.4 CUDA Certification

The CUDA kernel certification suite (`crates/xlog-cuda-tests/`) provides systematic coverage of all GPU kernel operations. On current `main`, the statistics in Table 9 hold.

Table 9: CUDA certification suite statistics (v0.6.1, April 2026).

| Metric | Value |
|---|---|
| Total tests | 206 |
| Categories | 33 (C01–C25 infrastructure + G01–G08 GPU-specific) |
| Cert modes (v0.6.1) | legacy/default; runtime+recorded; runtime+recorded+CSM |
| Latest full certification | 100% (206/206) in each mode |
| Kernel modules covered | 22 |

The 25 infrastructure categories (C01–C25) cover toolchain validation, launch configuration, pointer bounds, memory spaces (global, shared, local), synchronization, warp-level operations, control flow divergence, atomics, floating-point semantics, integer operations, determinism, async pipelines, caching, host-device transfer, multi-stream execution, multi-GPU, hardware stress, algorithmic correctness, blind spots, edge matrices, and float filter semantics (`docs/certification/2026-01-22-v0.4.0-alpha-certification-report.md`). The 8 GPU-specific categories (G01–G08) validate circuit forward evaluation, circuit backward (gradient) kernels, weight injection, transfer efficiency, circuit caching, PTX robustness, SAT/CDCL solving, and device-count invariants (`docs/architecture/cuda-certification.md`). The 22 covered runtime kernel modules correspond to the staged runtime CUDA kernel set plus the build-time ILP-family modules documented in the certification architecture notes. Because the public repository does not rely on hosted GPUs, the full certification suite is exercised through GPU-backed local release validation (`scripts/validate_release_gpu.sh -mode release`) and the manual self-hosted `cuda-ci` workflow; the always-on push/PR CI remains non-GPU and covers formatting, shell/workflow linting, package metadata, and no-GPU build checks.

## 9 Related Work

xlog draws on and extends five lines of research: the tradition of logic programming languages, GPU-accelerated Datalog evaluation, probabilistic logic programming, GPU SAT solving, and differentiable inductive logic programming. We survey each in turn, positioning xlog's contributions relative to the state of the art.

### 9.1 Logic Programming Languages

xlog sits in a long line of logic programming languages but is the first to target GPU execution of the full language surface. Prolog [15] is the canonical universal logic language, offering unification and extensive metaprogramming, running on a CPU tree-walking interpreter. Mercury [16] introduced strong typing and mode analysis to logic programming, compiling to optimized native code; xlog inherits the typed-predicate idea but targets device kernels rather than CPU code. Soufflé [17] is a typed Datalog dialect that compiles to efficient C++ and is widely used for program analysis; it is CPU-resident. Answer-set programming with clingo [18] provides nonmonotonic reasoning via answer-set semantics on CPU grounders and solvers; xlog is not an ASP system, but its stratified semantics and probabilistic extensions intersect ASP applications. Probabilistic logic languages such as ProbLog [12] and DeepProbLog [1] express weighted facts and neural predicates as first-class syntactic constructs, but their reasoners run on the CPU while neural components run on the GPU. xlog unifies all of these—typed logic, Datalog evaluation, probabilistic facts, neural predicates—as one language, compiled and executed entirely on the GPU.

### 9.2 GPU Datalog

Prior GPU-native Datalog systems—gpulog [3] and vflog [4]—target the Datalog evaluation backend that xlog also provides (Section 4), but do not surface a language of xlog's breadth (typed predicates, UDFs, modules, probabilistic and neural extensions). GPUlog ported bottom-up fixed-point computation to CUDA and reported approximately 45× speedups over equivalent CPU Datalog engines on transitive-closure benchmarks. VFLog advances this line with a vertically-fused, column-oriented kernel design that avoids materializing intermediate relations, achieving up to 200× speedups over CPU column engines on recursive graph queries. More recently, mnmgDatalog [19] extends GPU Datalog to multi-node, multi-GPU clusters, distributing semi-naive evaluation across devices via partitioned fact stores and all-to-all delta exchanges. All three systems target deterministic Datalog: they implement relational algebra operators (join, project, deduplicate) on the GPU and accelerate fixed-point iteration, but none supports probabilistic semantics, weighted model counting, or gradient computation over derived facts. xlog shares VFLog's commitment to columnar storage and kernel fusion for relational operators, but extends the GPU-resident execution model to probabilistic and differentiable reasoning, maintaining a single address space from semi-naive evaluation through circuit-based inference to neural-symbolic gradient propagation.

### 9.3 Probabilistic Logic Programming

ProbLog2 [12] established the modern pipeline for exact probabilistic inference in logic programs: ground the relevant program, encode provenance as a propositional formula, compile to a tractable target (d-DNNF or SDD), and evaluate weighted model counts over the compiled circuit. DeepProbLog [1] extends ProbLog2 by replacing selected probabilistic facts with the outputs of neural networks, enabling end-to-end gradient-based training of neural predicates within a logical framework. NeurASP [2] takes a related approach, integrating neural network outputs as probability annotations on answer-set programs and using stable-model semantics for inference. These systems demonstrate that combining neural perception with symbolic reasoning yields strong performance on tasks requiring both pattern recognition and logical structure. However, all three perform symbolic inference on the CPU: ProbLog2 invokes the D4 or c2d [20] compiler as a

host-side subprocess, DeepProbLog inherits this pipeline and adds a Python-level training loop that shuttles gradients between CPU logic and GPU tensors, and NeurASP relies on CPU-bound answer-set solving. The checked-in DeepProbLog baseline artifact in this repository is qualitative rather than a clean head-to-head benchmark: it records a 2:31:07 full-train run at 23.24% held-out addition accuracy and a smaller aborted run, while also noting protocol mismatches versus XLOG's archived measurements (`examples/neural/results/track_a/20260216T145409Z_-track_a_dev/comparisons/mnist_vs_deepproblog.json`). xlog eliminates the host/device bottleneck by compiling d-DNNF circuits into GPU-resident XGCF format with compile-once/evaluate-many semantics, executing both forward weighted model counting and backward gradient propagation as level-parallel CUDA kernels without host transfers during training iterations.

### 9.4 GPU SAT

ParaFROST [5] is a GPU-accelerated CDCL SAT solver that parallelizes clause database simplification—bounded variable elimination, subsumption, and self-subsuming resolution—on the GPU while retaining sequential conflict-driven search on the CPU. It is a complete solver with proof-generation capabilities, competitive with MiniSat and CaDiCaL on industrial benchmarks. FastFourierSAT [21] takes a different approach, reformulating SAT as continuous optimization over Fourier coefficients and applying local search on the GPU; it is an incomplete solver suited to satisfiable instances but unable to certify unsatisfiability. Both systems are standalone solvers designed for the SAT competition setting, not embedded within a logic programming pipeline. xlog's GPU CDCL module (Section 5, Stage 4) shares ParaFROST's strategy of offloading clause manipulation to device kernels, but integrates the solver as a verification component within the knowledge compilation pipeline—checking circuit-formula equivalence via two UNSAT proofs, computing minimal unsatisfiable cores, and providing proof certificates that ensure the correctness of compiled circuits.

### 9.5 Differentiable ILP

Differentiable inductive logic programming (dILP) [6] demonstrates that first-order rule induction can be cast as a differentiable optimization problem: candidate rules are scored by soft forward-chaining over all possible rule instantiations, and gradient descent adjusts rule weights to maximize coverage of positive examples. The approach is elegant but computationally demanding: forward-chaining materializes a dense tensor over the full rule space at each step, scaling cubically with the number of constants in the domain. Subsequent work has reduced this cost through pruning and approximate materialization, but implementations remain CPU-based and struggle beyond a few hundred constants. xlog's dILP subsystem (Section 6) replaces dense rule materialization with sparse GPU mask computation: candidate rules are represented as sparse bitmasks over the ground-atom space, credit assignment is performed entirely on-device via scatter-gather operations, and a six-gate promotion pipeline controls rule acceptance. This sparse, GPU-resident formulation avoids the cubic blowup of dense materialization while preserving differentiability, enabling rule induction over domains with thousands of constants.

### 9.6 Positioning

Each system surveyed above excels in its niche. GPUlog and VFLog demonstrate that relational fixed-point computation maps naturally to GPU architectures. ProbLog2 and DeepProbLog prove that knowledge compilation provides a principled bridge between logic and learning. ParaFROST shows that clause-level parallelism yields practical SAT speedups. dILP establishes that rule induction is amenable to gradient-based optimization. xlog unifies these capabilities on a single GPU-resident platform with zero-copy ML interop—not claiming superiority on any single axis, but providing the integrated stack. By keeping fact stores, compiled circuits, solver state, and gradient buffers in device memory throughout the reasoning pipeline, xlog eliminates the host-device transfer overhead that fragments existing multi-system approaches, offering practitioners a single engine where deterministic evaluation, probabilistic inference, SAT verification, and differentiable learning compose without leaving the GPU.

## 10 Limitations and Future Work

### 10.1 Current Limitations

**NVIDIA GPU required.** xlog targets CUDA exclusively. The entire execution stack—from hash-join kernels and radix sort to Monte Carlo sampling and knowledge compilation—is written in CUDA C. There is no OpenCL, Metal, or ROCm backend. This is a direct consequence of the GPU-native design philosophy: rather than abstracting over heterogeneous devices behind a portability layer, xlog commits to a single GPU programming model to exploit CUDA-specific features such as cooperative groups, warp-level primitives, and unified virtual addressing. Users without an NVIDIA GPU cannot run xlog.

**All data must fit in GPU memory.** Relations, indices, circuit buffers, and Monte Carlo sample arrays are allocated entirely on-device. There is no out-of-core spilling mechanism; a program whose working set exceeds GPU VRAM will fail with a `RESOURCE_EXHAUSTED` error rather than degrade silently. For most Datalog workloads this is not a bottleneck—a 24 GB GPU holds billions of 8-byte tuples—but large-scale knowledge graphs or high-sample-count MC inference can hit the ceiling.

**Python batch-query helpers are typed but not float-complete.** The Python helper APIs—`batch_fact_membership`, `batch_fact_membership_device`, `batch_tagged_credit`, and `batch_tagged_credit_device`—now perform schema-aware uploads for `u32`, `u64`, `i32`, `i64`, and `bool` columns, so wide integer IDs no longer truncate silently. However, these helper paths still reject floating-point batch uploads (`f32`/`f64`) at the binding layer. The underlying engine and DLPack relation-import paths remain typed; this is a convenience-API gap rather than a core execution limit.

**Differentiable ILP is beta.** The dILP subsystem—structure learning via gradient-based rule search—shipped in v0.4.0 and received significant reliability and performance work in v0.5.0–v0.6.1. It remains beta: the search space is restricted to definite programs, convergence is sensitive to learning-rate schedules, and the API surface may change in minor releases.

**No formal head-to-head benchmarks.** xlog's probabilistic inference pipeline occupies similar territory to DeepProbLog, but no controlled comparison on identical programs and datasets exists. The baseline experiments in Section 8 use different dataset sizes and evaluation protocols. Claims of

speedup relative to CPU-based probabilistic logic systems remain informal until such benchmarks are published.

### 10.2 Future Work

The following directions are planned but **not yet implemented** in any released version of xlog.

**Epistemic logic programming (xlog-elp).** The EIR (Epistemic Intermediate Representation) slot is reserved in the IR stack (Section 2), but no runtime code has been written. The planned design supports world views under FAEEL (Founded Autoepistemic Equilibrium Logic) semantics with modal operators K (known in all belief sets) and M (possible in some belief set), evaluated via a Generate–Propagate–Test algorithm on GPU. This is targeted for v0.7.0 (`ROADMAP.md`); v0.6.0 was instead reserved for the stream-safe GPU runtime described in Section 2.

**Out-of-core execution.** For programs whose materialized relations exceed GPU memory, a streaming execution mode would partition relations across host and device, paging tiles through GPU memory in fixpoint-iteration order. This would remove the single-GPU-memory ceiling described above, at the cost of added host-device transfer overhead. Targeted for v0.8.0 alongside multi-GPU partitioning (`ROADMAP.md`).

**Magic sets transformation.** Top-down query-directed rewriting would restrict bottom-up evaluation to tuples reachable from a given query, potentially yielding orders-of-magnitude reduction in materialized facts for selective queries. The optimizer currently performs predicate pushdown and cost-based join planning (Section 4) but does not yet rewrite the program itself.

**Multi-GPU partitioned evaluation.** Inspired by mnmgDatalog's radix-hash partitioning and GPU-aware all-to-all shuffle, a multi-GPU backend would distribute relations across devices and coordinate join and union operations via NVLink or PCIe. Partitioning kernels are described in the roadmap but unimplemented.

**Incremental parsing.** The pest-based PEG parser currently re-parses the entire program on every change. An incremental mode would re-compile only modified rules, enabling tighter interactive feedback loops in notebook and REPL workflows.

## References

[1] Robin Manhaeve et al. "DeepProbLog: Neural Probabilistic Logic Programming". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[2] Zhun Yang, Adam Ishay, and Joohyung Lee. "NeurASP: Embracing Neural Networks into Answer Set Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*. 2020.

[3] Yihao Sun et al. "Optimizing Datalog for the GPU". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2025. DOI: 10.1145/3669940.3707274.

[4] Yihao Sun et al. "Column-Oriented Datalog on the GPU". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 15177–15185. DOI: 10.1609/aaai.v39i14.33665.

[5] Muhammad Osama, Anton Wijs, and Armin Biere. "SAT Solving with GPU Accelerated Inprocessing". In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12651. Lecture Notes in Computer Science. Springer, 2021, pp. 133–151. DOI: 10.1007/978-3-030-72016-2_8.

[6] Richard Evans and Edward Grefenstette. "Learning Explanatory Rules from Noisy Data". In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 1–64.

[7] Jean-Marie Lagniez and Pierre Marquis. "An Improved Decision-DNNF Compiler". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*. 2017.

[8] *DLPack: Open In Memory Tensor Structure*. https://github.com/dmlc/dlpack. Accessed 2026.

[9] *Apache Arrow: Cross-Language Development Platform for In-Memory Data*. https://arrow.apache.org. Accessed 2026.

[10] *cudarc: Safe Rust Wrapper around CUDA*. https://github.com/coreylowman/cudarc. Accessed 2026.

[11] *PyO3: Rust Bindings for Python*. https://pyo3.rs. Accessed 2026.

[12] Daan Fierens et al. "Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas". In: *Theory and Practice of Logic Programming* 15.3 (2015), pp. 358–401.

[13] Grigori S. Tseitin. "On the Complexity of Derivation in Propositional Calculus". In: *Studies in Constructive Mathematics and Mathematical Logic* (1968).

[14] João P. Marques-Silva and Karem A. Sakallah. "GRASP—A New Search Algorithm for Satisfiability". In: *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1996.

[15] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[16] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. "The Execution Algorithm of Mercury, an Efficient Purely Declarative Logic Programming Language". In: *Journal of Logic Programming* 29.1-3 (1996), pp. 17–64.

[17] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. "Soufflé: On Synthesis of Program Analyzers". In: *Computer Aided Verification (CAV)*. Springer, 2016, pp. 422–430.

[18] Martin Gebser et al. "Multi-shot ASP Solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.

[19] Ahmedur Rahman Shovon et al. "Multi-Node Multi-GPU Datalog". In: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*. 2025. DOI: 10.1145/3721145.3730431.

[20] Adnan Darwiche. "New Advances in Compiling CNF into Decomposable Negation Normal Form". In: *Proceedings of the 16th European Conference on Artificial Intelligence (ECAI)*. 2004.

[21] Yunuo Cen, Zhiwei Zhang, and Xuanyao Fong. *Massively Parallel Continuous Local Search for Hybrid SAT Solving on GPUs*. arXiv:2308.15020. 2023.